Maintaining the ODEKA Website

A Guide for Research Associates

ODEKA A.S.B.L

August 17, 2026

Contents

Part I

Getting Started with Quarto

1 Introduction

This guide is for research associates and other ODEKA staff who need to update the ODEKA website (`https://odeka-asbl.github.io`) — adding a news post, a job listing, a new team member, or a new project, for example.

The website is built with a tool called **Quarto**. You do not need to know how to code to use it: pages are written mostly in plain text (with a little bit of HTML sprinkled in for layout), and

Quarto turns that text into the actual website.

This first part of the guide covers two things:

1. Setting up your computer with the tools you need (Section 2).
2. How Quarto works in general — the concepts you need to understand before touching this specific website (Section 3).

Part II covers the specifics of *this* website: its folder structure, its custom design system, and step-by-step instructions for common tasks (adding a news post, adding a team member, etc.).

2 Setting Up Your Computer

You only need to do this once. If a tool is already installed on your computer, you can skip that step.

2.1 Install Visual Studio Code

Visual Studio Code (**VS Code**) is the text editor we use to write and edit the website's files.

1. Go to `https://code.visualstudio.com`.
2. Download the version for your operating system (Mac, Windows, or Linux) and run the installer.
3. Open VS Code once to confirm it launches correctly.

2.2 Install Quarto

Quarto is the actual engine that converts our source files into the website.

1. Go to `https://quarto.org/docs/get-started/`.
2. Download and run the installer for your operating system.
3. To confirm it installed correctly, open a terminal (on Mac: Terminal app; on Windows: Command Prompt or PowerShell) and type:

   ```
   quarto check
   ```

   This should print a summary confirming Quarto is installed, along with the version number, without any errors.

2.3 Install the Quarto Extension for VS Code

This extension gives you syntax highlighting, a "Preview" button, and other conveniences directly inside VS Code.

1. Open VS Code.
2. Click the Extensions icon in the left-hand sidebar (it looks like four small squares), or press `Cmd+Shift+X` (Mac) / `Ctrl+Shift+X` (Windows).

3. Search for **Quarto**.
4. Install the extension published by **Quarto**.

Once installed, opening any `.qmd` file in VS Code will show a "Preview" button (a small icon with a magnifying glass, usually in the top-right corner of the editor) that opens a live preview of that page.

2.4 Install Git and Get Access to the Repository

The website's files live in a **Git repository** hosted on **GitHub**, at a URL your supervisor will share with you. Git is the tool that tracks changes to the files and lets you publish ("push") your edits so they show up on the live website.

1. Create a free account at https://github.com if you don't already have one, and ask your supervisor to give your account access to the repository.
2. Install Git:
   - **Mac**: open Terminal and type `git -version`. If Git isn't already installed, macOS will prompt you to install the Xcode Command Line Tools, which include it.
   - **Windows**: download and install Git from https://git-scm.com/downloads.
3. (Optional but recommended for beginners) Install **GitHub Desktop** (https://desktop.github.com), a graphical app that lets you commit and push changes without typing Git commands directly. VS Code also has built-in Git support (the "Source Control" icon in the sidebar) if you prefer to stay in one app.

2.5 Connecting VS Code to Your GitHub Account

Before you can push changes, Git needs to know it's really you. Note that GitHub no longer lets you sign in for this using your regular account password — you'll instead use one of the two methods below.

Option A: Sign in through VS Code (recommended). This is the easiest method and involves no typing of tokens at all.

1. In VS Code, click the **Source Control** icon in the left-hand sidebar (it looks like a branching line).
2. Click **Sign in** (or, if you don't see it there yet, wait until you perform your first push — VS Code will prompt you automatically at that point).
3. Your web browser will open a GitHub page asking you to log in (username and password there, on GitHub's own site) and then **authorize Visual Studio Code** to access your account.
4. Click **Authorize**. You'll be sent back to VS Code, which is now connected to your GitHub account. You will not be asked again on this computer.

If you installed GitHub Desktop instead, the equivalent step is: **GitHub Desktop** → **Settings** (Mac) or **Options** (Windows) → **Accounts** → **Sign in to GitHub.com**, which opens the same kind of browser sign-in window.

Option B: Personal Access Token (if you work from the terminal). If you type Git commands directly in a terminal rather than using VS Code's Source Control panel or GitHub Desktop, the first time you `git push` you'll be prompted for a username and password. Enter your GitHub username as usual, but for the password, you must use a **Personal Access Token** (a long, randomly generated code) instead of your real password — typing your real password here will simply fail.

To create one:

1. Go to https://github.com/settings/tokens.
2. Click **Generate new token → Generate new token (classic)**.
3. Give it a name (e.g. "ODEKA laptop"), set an expiration date, and check the box next to **repo**.
4. Click **Generate token** at the bottom of the page, then **copy the token immediately** — GitHub will only show it to you once.
5. The next time Git asks for a password, paste this token instead. Your computer will normally remember it after that, so you won't need to repeat this often.

Unless you have a specific reason to work from the terminal, Option A is simpler and is what we recommend.

2.6 Clone the Repository

"Cloning" means downloading a working copy of the website's files onto your computer.

1. Open a terminal and navigate to the folder where you want to keep the project, e.g.:

```
cd Documents
```

2. Clone the repository (replace the URL with the one your supervisor gives you):

```
git clone https://github.com/odeka-asbl/odeka-asbl.github.io.git
```

3. Open the resulting folder in VS Code: **File → Open Folder...** and select it.

You now have everything you need on your machine. The remainder of this document explains how Quarto itself works.

3 Quarto: The General Principles

3.1 What Quarto Is

Quarto is a **static site generator**. That means: you write content in simple source files, and Quarto "renders" (builds) those files into plain HTML pages — the actual files a web browser displays. Nothing runs on a server when someone visits the site; every page is just a pre-built HTML file sitting in a folder, served as-is. This makes the site fast, simple to host (we use GitHub Pages, which is free), and hard to break in unpredictable ways.

The two building blocks you'll work with constantly are:

- **`.qmd` files** — these are the source files for each page. "qmd" stands for Quarto Markdown. You write and edit these.
- **The `docs/` folder** — this is where Quarto puts the finished HTML files after rendering. This is what actually gets published to the live website. **You should never hand-edit files inside `docs/`** — they get overwritten every time the site is rendered again. Always edit the `.qmd` source files instead.

3.2 Markdown, Briefly

Most of the text content on the site is written in **Markdown**, a simple way to format text using plain characters instead of a toolbar. The basics:

```
# This is a big heading
## This is a smaller heading

Regular paragraph text goes here. You can make text **bold**
or *italic*. You can add a [link](https://example.com), or an
image: ![alt text](path/to/image.jpg)

- A bullet point
- Another bullet point
```

Our website also mixes in small pieces of raw HTML directly inside the `.qmd` files (for things like banners, custom layouts, and buttons) — this is normal and something we'll cover in Part II. You don't need to know HTML deeply, but you will encounter it.

3.3 The YAML Frontmatter

Every `.qmd` file starts with a block between two lines of three dashes (`---`). This is called the **frontmatter**, written in a format called **YAML**, and it controls settings for that specific page: its title, whether it has a table of contents, and so on.

```
---
title: "About us"
toc: false
---

The rest of the page's content starts here, after the
closing '---'.
```

3.4 The Project-Wide Configuration File

At the root of the project sits a single file, `_quarto.yml`, which controls settings for the *entire* website rather than one page. This is where the navigation bar, the site title, and the overall output folder are defined. For example, a simplified version of ours looks like:

```
project:
```

```
  type: website
  output-dir: docs

website:
  title: "ODEKA"
  navbar:
    left:
      - href: 01_about/about.qmd
        text: About us
      - href: 02_people/people.qmd
        text: People

format:
  html:
    theme: cosmo
    css: styles.css
```

Notice how each navbar entry simply points to a `.qmd` file and gives it a label. Adding a new top-level page to the site generally starts with adding an entry here.

3.5 Rendering the Site

"Rendering" is the act of converting your `.qmd` source files into the finished HTML in `docs/`. You do this from the terminal, from the project's root folder:

```
quarto render
```

This rebuilds the *entire* site. If you only changed one page and want a faster rebuild, you can render just that file:

```
quarto render 04_news/index.qmd
```

You need to render (and then commit and push, see Section 3.7) before your changes actually appear on the live website.

3.6 Previewing Your Changes Live

Rendering the whole site every time you make a small edit is slow. Instead, while you're actively working on a page, use **preview mode**, which opens a local copy of the site in your browser and automatically refreshes it every time you save a file:

```
quarto preview
```

This will print a local address (something like `http://localhost:4848`) and normally opens it in your browser automatically. Leave this command running in the terminal while you work; press `Ctrl+C` in the terminal to stop it when you're done.

If you installed the Quarto VS Code extension mentioned earlier, you can also just click the **Preview** button at the top of an open `.qmd` file, which does the same thing without needing the terminal.

3.7 Publishing Your Changes

The live website is served directly from the `docs/` folder in the GitHub repository. Publishing a change means three steps:

1. **Render** the site (or the page you changed), so `docs/` is up to date:

   ```
   quarto render
   ```

2. **Commit** your changes — this saves a labeled snapshot of what changed, both in the source files and in `docs/`:

   ```
   git add .
   git commit -m "Add news post about the new field team"
   ```

3. **Push** your commit to GitHub, which is what actually makes it live (usually within a minute or two):

   ```
   git push
   ```

If you're using GitHub Desktop or VS Code's Source Control panel instead of typing Git commands, these same three steps exist as buttons: review your changed files, write a short commit message, click **Commit**, then click **Push**.

3.8 How a Quarto *Website* Project Is Organized

A single Quarto document is just one file. A Quarto *website* project — which is what we have — is a folder containing many `.qmd` files (one per page), plus:

- **Shared styling**: a single `.css` file (ours is `styles.css`) that controls fonts, colors, spacing, and layout across every page, so the whole site looks consistent.
- **Listings**: some pages (like News or Jobs) don't list their content by hand. Instead, they automatically generate a list of posts from a folder, sorted by date, based on settings in that page's frontmatter. Adding a new post is often just a matter of adding a new file to the right folder — the listing picks it up automatically.
- **Includes**: small snippets of HTML or JavaScript that get inserted into every page automatically (for example, code that powers a hover effect in our navigation bar). These live in their own files and are referenced from `_quarto.yml`.
- **Images and other assets**: stored alongside the pages that use them, referenced by file path from within the `.qmd` files.

3.9 A Typical Editing Workflow, Start to Finish

Putting it all together, here is what a routine edit looks like:

1. Open the project folder in VS Code.
2. Open the relevant `.qmd` file and start editing.

3. Run `quarto preview` (or click the Preview button) to watch your changes as you make them.
4. Once you're happy with the result, stop the preview, run `quarto render` to rebuild the whole site cleanly, and skim a few pages to make sure nothing else broke.
5. Commit and push your changes (Section 3.7).
6. Check the live website after a minute or two to confirm your change appears correctly.

4 Editing the Website with Claude Code

Everything above describes doing the work by hand: opening a file, editing Markdown and HTML yourself, running the commands yourself. That's essential to understand, but it isn't the only way to work. **Claude Code** is an AI coding assistant from Anthropic that runs inside VS Code (or a terminal) and can read the whole project, make the edits itself, run `quarto render`, and even commit and push — all from plain-English instructions you give it, like "add a new job listing for a Research Intern position, same format as the existing ones."

Starting in Part II, this guide will show most tasks *two* ways: how to do them yourself, step by step, and a suggested prompt you could give Claude Code instead to have it done for you in seconds. Understanding the manual steps is still worthwhile — it means you can judge whether Claude Code's changes look right and fix small things yourself when needed — but for most day-to-day updates, describing what you want is often faster than doing it by hand.

4.1 Installing Claude Code in VS Code

1. Open VS Code and click the Extensions icon in the sidebar (or press `Cmd+Shift+X` / `Ctrl+Shift+X`).
2. Search for **Claude Code** and install the extension published by Anthropic.
3. A new Claude icon will appear in the sidebar. Click it to open the Claude Code panel.
4. You'll be prompted to sign in. This uses the same kind of browser-based sign-in as GitHub in Section 2.5: click **Sign in**, log into your Claude account in the browser window that opens, and authorize VS Code.
5. Using Claude Code requires an active Claude plan (Pro or Max) or billing set up on the Anthropic Console. Ask your supervisor if ODEKA already has an account you should use, or if you need to set up your own at `https://claude.ai`.

4.2 Working with Claude Code

1. Make sure the ODEKA website folder is the one open in VS Code (**File → Open Folder...** if it isn't) — Claude Code works on whichever project is currently open.
2. In the Claude Code panel, type what you want in plain English, for example:

   > *"Add a news post announcing that ODEKA hired a new field coordinator, dated today. Use the same format and image size as the other news posts."*

3. Claude Code will make the edits, and can also run `quarto render`/`quarto preview` for you and describe what it changed. Review its summary and the changed files before moving on.

4. Ask it to show you a preview, or open one yourself (Section 3), to confirm the result looks right.
5. You can ask Claude Code to commit and push the change too ("commit this and push it"), or do that step yourself once you're happy with it — either is fine.

A few habits worth keeping. Be specific about which page and what you want (naming the exact page, section, or person makes for a better result than a vague request). Always look over what changed before pushing, the same way you'd proofread something before sending it. And if a result isn't quite right, just say so in plain language — "move that a bit to the left," "make the text smaller" — the same way the rest of this guide describes fixing things by hand.

4.3 What Claude Code Already Knows: `CLAUDE.md`

A conversation with Claude Code is not remembered afterward. Close the window, and the next session — even yours, even five minutes later — starts from nothing except the files in the project. That's normally fine, since Claude Code can read the code itself, but some things aren't visible just from reading the code once: a bug that only shows up on the live site and not while previewing locally, a design choice that was tried and deliberately rejected, a CSS rule that looks redundant but isn't. Losing that kind of hard-won context between sessions is wasteful — it means re-discovering the same problem twice.

To fix this, the repository has a file at its root named `CLAUDE.md`. Claude Code reads it automatically, every time, at the start of any session opened in this folder — on any computer, for anyone. It exists purely to carry exactly this kind of knowledge forward: known traps (for example, an image that displays fine locally but silently fails once published, because of a filename case mismatch that only matters on the live server), the reasoning behind non-obvious choices, and pointers to this guide for the step-by-step version of a task.

You don't need to write it yourself. If Claude Code hits something tricky while helping you — a bug with a non-obvious cause, a setting that has to be right in three different files at once — it should be the one updating `CLAUDE.md`, not you. A reasonable habit: after Claude Code fixes something confusing, ask it directly, "is this worth adding to CLAUDE.md?" If a future problem turns out to be something Claude Code already ran into before but never wrote down, that's a sign to ask it to add a note next time.

How this differs from this guide. `CLAUDE.md` is written *for Claude Code* — terse, technical, assumes familiarity with the code. This document is written *for you* — the person doing the work, whether by hand or by asking Claude Code. They're meant to stay in sync but serve different readers: if you make a structural change to how a page works, both may need updating (Section 11.5 says more about keeping this document itself current).

Part II

The ODEKA Website, Page by Page

This part walks through every page of the site and the specific edits you'll actually make: updating a bio, adding a project, publishing a news post, posting a job, or swapping out a photo. Each task is shown the manual way first, then — where it's genuinely faster — as a Claude Code prompt (Section 4). Reading the manual version at least once is worth it even if you always use Claude Code afterward: it's what lets you tell whether what Claude Code did is actually right.

A general note before starting: after *any* edit described below, the workflow is always the same three steps from Part I — render (or preview while you work), check it, then commit and push (Section 3.7). This part won't repeat that every time.

5 The People Page

The People page (`02_people/people.qmd`) is one long file. It's organized into named sections (Senior Management, PhD Students, Past Research Associates, and so on), each containing a run of individual entries. There's no listing magic here like on News or Jobs — every person is hand-written directly in this one file, in the order they appear on the page.

5.1 How an Entry Is Built

Every person has the same two-part shape: a small square photo, and an info block next to it with their name, title, and (optionally) a short bio and a link. Most entries look like this:

```
<div style="clear: both;">
</div>
<br>

<div class="person-photo">
  <img src="faces/lastname.jpg" alt="Full Name">
</div>

<div class="person-info">
<div class="person-name">
  Full Name
</div>

<div class="person-title">
  Their Title
</div>

<p>Optional one- or two-sentence bio.

<div class="person-link">
<a href="https://example.com" target="_blank">
  Profile Link Text
</a>
```

```
</div>
</div>
```

A few sections list people with no photo at all — currently "Past Research Interns", and "Research Interns" should follow the same pattern once people are added there (it's empty for now). Those use a slightly different wrapper — the photo `<div>` is simply skipped, and `person-info` gets a second class:

```
<div style="clear: both; margin-bottom: 38px;">
</div>

<div class="person-info person-info--no-photo">
<div class="person-name">
  Full Name
</div>
<div class="person-title">
  Their Title
</div>
<div class="person-link">
<a href="https://example.com" target="_blank">
  LinkedIn Profile
</a>
</div>
</div>
```

If someone doesn't have a photo yet, use `faces/placeholder.jpg` rather than leaving it out — every entry on the page currently has an image, and the placeholder (a plain silhouette icon) is what keeps the layout consistent until a real photo is added.

5.2 Editing Someone's Information

Open `02_people/people.qmd`, search for their name, and edit the text directly — the name, title, bio paragraph, or link text/URL are all plain text inside the block shown above. To swap their photo, replace the file at `02_people/faces/whatever.jpg` with the new image *using the same filename* (simplest option), or add a new image file and update the `src="..."` path to match.

> *Ask Claude Code: "On the People page, update Jon Weigel's title to 'Scientific Director' and change his bio to say he's now at [institution]."*

5.3 Adding a Person

1. Find the section they belong in, and find another entry in that same section to use as a template.
2. Copy that whole entry (from its opening `<div style="clear: both...` down to the closing `</div>` of `person-info`) and paste it either right before or right after it, depending on where the new person should sit.
3. Update the name, title, bio, link, and photo path. If you don't have a photo yet, point it at `faces/placeholder.jpg`.

4. If the section is ordered a particular way (PhD Students is alphabetical by family name; Past Research Associates run newest-to-oldest), place the new entry in the position that keeps that order.

   Ask Claude Code: "Add a new person to the PhD Students section of the People page: [Name], PhD Student, no bio yet, placeholder photo. Keep the section in alphabetical order by family name."

5.4 Removing a Person

Delete their entire block, from the `<div style="clear: both...` spacer immediately before their photo down through the closing `</div>` of `person-info`. Be careful with the closing `</div>` tags in particular — there are several nested divs in each entry, and deleting one too many (or too few) will silently break the layout of the *next* person's entry rather than produce an error. Render the page afterward and check that both the deleted person is gone and the people around them still look normal.

Ask Claude Code: "Remove Jean-François Coté's entry from the Past Research Associates section of the People page."

5.5 Adding a New Section

Copy this pattern, placing it wherever the new section should sit between two existing ones:

```
<!-- ============================ -->
<!-- ============================ -->
<!-- ============================ -->
<div class="about-section-divider"></div>

# New Section Title {.people-eyebrow-heading}

One-sentence description of who belongs in this section.
<!-- ============================ -->
```

Then add person entries underneath it as described above. Two things to know:

- The heading text becomes an entry in the "on this page" box on the right automatically — nothing else to configure.
- If the new heading's text is identical to another heading already on the page (this has come up before, e.g. two sections both wanting to be called "Past Directors"), add an explicit anchor to keep their links from colliding: `# New Section Title {#a-unique-id .people-eyebrow-heading}`.

  Ask Claude Code: "Add a new section to the People page called 'Visiting Researchers', right after Scientific Advisory Committee, with the description 'ODEKA hosts visiting researchers who contribute to our work during their stay.' Leave it empty for now."

6 The Projects Page

Unlike People, Projects uses Quarto's **listing** system: each project is its own file in `03_projects/posts/`, and both the Projects page and the homepage carousel are automatically generated from those files based on a few frontmatter fields. You never edit a list of projects by hand — you add, edit, or delete the individual project files, and the listings update themselves.

6.1 The Fields That Matter

Every project file's frontmatter looks like this:

```
---
title: "Project Name"
author:
  - Person One
  - Person Two
date: 2020-01-01
display date: "2020-2023"
image: ../images/projectN.jpg
description: "One-sentence summary shown on the card."
status: ongoing
type: major
about:
  template: marquee
---
<style>
header.quarto-title-block .description,
.quarto-title .description,
.quarto-title-meta .description,
p.description {
  display: none !important;
}
</style>

[ Back to Projects](../index.qmd){.breadcrumb}

The actual page content goes here.
```

The `marquee` template (the same one News posts use) is what puts a large banner image at the top of the page instead of a small sidebar photo — the `<style>` block right after it is required for the same reason it's required on News posts (Section 7): without it, the `description` field shows up twice.

Two fields drive everything about where a project shows up, and they are independent of each other:

- `type`: either `major` or `other`. This decides which section of the *Projects page itself* the project appears under ("Major Projects" or "Other Projects").
- `status`: currently `ongoing` or anything else (e.g. `completed`). This only affects the little "ONGOING" badge shown on the card — *except* for one important consequence below.

6.2 The Homepage Carousel Is Pickier Than the Projects Page

The Projects page shows every project of a given `type`, regardless of status. The **homepage** carousel ("Ongoing major projects") is more restrictive: it only shows projects where `type: major` *and* `status: ongoing` are *both* true. This means:

- Marking a major project's status as anything other than `ongoing` removes it from the homepage immediately, but it stays exactly where it was on the Projects page.
- An `other`-type project will never appear on the homepage carousel, no matter its status.
- If you want a project to stop being featured on the homepage the moment it wraps up, changing its `status` field is the single edit that does it — nothing else needs to change.

6.3 Adding a Project

1. Create a new file in `03_projects/posts/`, e.g. `my-new-project.qmd`, using the template above.
2. Add a card image to `03_projects/images/` and point `image:` at it.
3. Set `type` and `status` deliberately, per the rules above.
4. Write the actual project page content below the frontmatter (see Section 11.2 for the current state of this — it's genuinely unresolved for existing projects too).

   Ask Claude Code: "Add a new major, ongoing project called '[Name]' to the Projects page, described as '[one sentence]'. Use [image file] as the card image."

6.4 Editing or Removing a Project

To edit, open its file in `03_projects/posts/` and change the frontmatter fields or the page content directly. To remove a project entirely, delete its `.qmd` file (and its image, if nothing else uses it) — it disappears from every listing automatically.

> *Ask Claude Code: "Change the Religion & Development project's status to completed" or "Delete the [project name] project."*

7 The News Page

News works the same way as Projects: one file per post in `04_news/posts/`, listed automatically.

7.1 Adding, Editing, or Removing a Post

A news post's frontmatter is simpler than a project's:

```
---
title: "Headline Goes Here"
author:
  - Author Name
date: "Month DD, YYYY"
description: "One-sentence summary."
image: ../images/newsYYYYMMDD.jpg
```

```
about:
  template: marquee
---
<style>
header.quarto-title-block .description,
.quarto-title .description,
.quarto-title-meta .description,
p.description {
  display: none !important;
}
</style>

[ Back to News](../index.qmd){.breadcrumb}

**A bold one-sentence lead-in, restating the summary.**

The rest of the story, in normal paragraphs.
```

That `<style>` block isn't decorative — without it, the `description` shows up twice (once as Quarto's own subtitle, once as your bold lead-in paragraph). Keep it in every new post.

To add a post: copy an existing file in `04_news/posts/`, rename it (the convention so far is `newsYYYYMMDD.qmd`), update the frontmatter and body, and add its image to `04_news/images/`. To edit one, just open and change it. To remove one, delete its file.

> *Ask Claude Code: "Add a news post dated today announcing [whatever happened]. Use the same format as the existing posts."*

7.2 Turning the News Section On and Off

If nobody is actively writing news, the whole section can be switched off — hidden from the navbar, removed from the homepage (replaced by a plain divider), and excluded from the built site — without deleting any of the actual posts. Turning it back on later restores everything exactly as it was, because nothing in `04_news/` itself is ever touched by the switch. It only takes a handful of small, coordinated edits across two files:

1. In `_quarto.yml`, find the block of comments marked `NEWS SECTION SWITCH` near the top, inside the project's `render:` list. Uncomment the single `"!04_news/**"` exclusion line inside it (the list already always excludes `CLAUDE.md` above it — leave that line alone).
2. A little further down in the same file, in `navbar: left:`, comment out the two lines for the News entry (marked with a `NEWS SWITCH` comment right above them).
3. In `index.qmd`, two more places are marked `NEWS SWITCH`:
   - Comment out the `news_carousel` listing definition in the frontmatter (this one matters — if you skip it, Quarto will dump the news carousel awkwardly at the very bottom of the homepage instead of hiding it).
   - Change the `{{< include ... >}}` line just above where "Latest News" used to be from `news_section_on.qmd` to `news_section_off.qmd`.

4. Run `quarto render`. That single render does everything: removes the News pages that were already built, updates the navbar, and swaps in the plain divider on the homepage.

To turn News back on, reverse the edits from steps 1–3 above (re-comment/uncomment the opposite way), then render again.

> *Ask Claude Code: "Turn the News section off" or "Turn the News section back on" — it knows to make all the coordinated edits together and re-render.*

See Section 11.3 for a note on *when* you might actually want to do this.

8 The Jobs Page

Jobs also uses one file per posting, in `05_jobs/posts/`, but the page layout is richer than News — it has a structured header (supervisors, dates, location, duration) above the actual posting text.

8.1 The Content Rule: Copy the Posting Verbatim

This is important enough to call out on its own: when you add a real job posting, the body text (Overview, Responsibilities, Qualifications, etc.) should be an exact, word-for-word copy of the actual posting — wherever it's really hosted (J-PAL/IPA, a university jobs board, wherever) — not a rewritten or summarized version. Only the *layout* (the section headers, the meta block at the top, colors, fonts) is ours; the wording is not. When you fetch the source page to copy from, copy the literal text, not a paraphrase of it.

One consequence of this: our postings are sometimes hosted on a recruiting platform (J-PAL/IPA's shared application system) as a matter of practicality, but **ODEKA is always the actual employer**. Keep that clear on the page — e.g. the short description under the title should say "We are hiring...", not name the hosting platform as the employer.

8.2 The Template

```
---
title: "Exact Job Title"
author:
  - Supervisor Name, Affiliation
date: "Month DD, YYYY"
description: "We are hiring a [role] to [one-sentence summary]."
language:
  title-block-author-single: "Supervisor"
  title-block-author-plural: "Supervisors"
  listing-page-field-author: "Supervisor"
---

<div class="job-page">

<div class="job-meta">
<div class="job-meta-item"><strong>Posted</strong>Month DD, YYYY</div>
<div class="job-meta-item"><strong>Supervisors</strong>Name One<br>Name Two</div>
<div class="job-meta-item"><strong>Location</strong>City, Country</div>
<div class="job-meta-item"><strong>Start Date</strong>Month D, YYYY</div>
<div class="job-meta-item"><strong>Duration</strong>N months</div>
```

```
</div>

[ View All Open Positions](../index.qmd){.breadcrumb}

### Overview
Copied verbatim from the real posting.

### Responsibilities
- Copied verbatim, as a bullet list.

### Qualifications
- Copied verbatim (required)
- Copied verbatim (desired)

### Supervisors
**Primary supervisors:** ...
**Secondary supervisors:** ...

### How to Apply
Copied verbatim, plus a plain-language note that ODEKA is the employer
if the original posting doesn't make that obvious.

<div class="job-apply">
<a class="about-button" href="https://actual-application-url"
   target="_blank">Apply for this position </a>
</div>

</div>
```

The `language:` block is what relabels Quarto's default "Author" heading to "Supervisor"/"Supervisors" — keep it in every posting. The section headers (`###`) can be adjusted to match whatever structure the real posting actually uses; they don't have to be exactly these five.

8.3 Adding, Editing, or Removing a Posting

Same pattern as News and Projects: add a new file in `05_jobs/posts/` for a new posting, edit a file directly to update one, delete a file to remove one (once it's filled or closed).

> *Ask Claude Code: "Here's a job posting URL: [link]. Add it to the Jobs page, copying the content verbatim, in our usual job-page format."*

9 Contacts and Legal Notice

These two are the simplest pages on the site — each is a single file (`06_contacts/contacts.qmd` and `07_legal_notice/legal_notice.qmd`) with plain text content, organized under `###` headings:

```
### Section Heading:
Whatever text or contact details go here.<br>
Use <br> for a line break within a section.
```

Just open the relevant file and edit the text directly — add, remove, or reword a `###` section however needed. The photo shown next to the text is set by the `image:` field in the frontmatter (`about: image: image1.jpg`); replace that file to change it.

10 Changing an Illustration Anywhere on the Site

Every image on the site is a plain file referenced by path, either in a page's frontmatter (`image: ...`) or directly in the page content (`<img src="...">`). To swap one out, you have two options:

1. **Simplest**: replace the file at that exact path with your new image, keeping the same filename. Nothing else needs to change.
2. Add a new image file and update the path that references it, if you'd rather keep the old one around or use a more descriptive filename.

10.1 Where Images Live, and How They're Named

Every page keeps its own images in its own folder, right next to the `.qmd` file that uses them — there is no single shared `images/` folder for the whole site. The pattern is consistent enough to state as a table:

What	Where, and how it's named
Page banner (top/bottom of a page)	`0N_folder/imageN.ext` (sequential)
Person photo	`02_people/faces/lastname.ext`
Project card image	`03_projects/images/projectN.ext`
News post image	`04_news/images/newsYYYYMMDD.ext`
Site-wide logos/brackets	root folder, `odeka_logo_*.png`

Stick to whichever pattern already applies to the page you're editing. This matters more than it might seem, for two reasons we've already hit in practice, not hypothetically:

- **File extension case has to match exactly.** One news post once linked to `news20220627.JPG` while the actual file on disk was `news20220627.jpg`. It worked by accident on a Mac (whose filesystem ignores case) and would have silently shown a broken image once deployed, since GitHub Pages' hosting does not. Always match the real filename's case exactly, and prefer lowercase extensions for anything new.
- **Unused images pile up fast if filenames drift.** A routine cleanup pass on this site once found — and removed — ten image files that nothing on the site referenced anymore, plus an entire folder of raw, never-used originals. That happens naturally when a new image doesn't follow the existing naming pattern closely enough to make it obvious which page it belongs to, or whether an old one it was meant to replace is still hanging around.

Following the table above — same folder as the page, same naming style as the images already there — is what keeps that from happening again. If you ever do add an image and later decide not to use it, delete it rather than leaving it behind.

10.2 Cropping and Sizing Conventions

- **Person photos** (`02_people/faces/`): square crops, centered on the face, saved at a reasonable resolution (the existing photos are roughly 400–800px square). They're displayed at

200×200px, so anything comfortably larger than that is fine.

- **Banners** (the full-width images at the top and bottom of most pages): wide landscape crops. Keep the subject roughly centered horizontally — the banner crops responsively as the browser window resizes, and content too close to the left or right edge can get cut off on narrower screens.
- **Card images** (Projects, News): roughly a 4:3 or similar landscape crop works best; they display inside a fixed-size card.

  Ask Claude Code: "Replace [person]'s photo on the People page with [new file], cropped the same way as the others."

11 What Remains To Be Done

This section is a snapshot of known gaps as of this writing — a starting checklist, not a complete one. Update or trim it as items get resolved.

11.1 People Page

- **Photos missing**: several Senior Management and Senior Enumerators entries (Junior Ntambue, Gilbert Tshiebue, Jacques Kazadi, Max Mbosho, Grégoire Ntumba, Bernard Makinda, Serge Nseya, André Kapumbu, Benjamin Badibanga, Théo Kalamba, Jean Kalala), plus most of the current PhD Students (Clotaire Boyer, Sarah Danner, Eva Davoine, Gabriel Granato, Nic Wicaksono), are still showing the placeholder silhouette. Arthur Laroche, the sixth current PhD student, already has a real photo — his entry is the model to follow for the rest. See Section 10.2 for cropping conventions once more real photos are ready.
- **PhD Students section is missing information**: for each current PhD student, we're missing the year they started working with ODEKA and their PhD institution — probably worth labeling the same way Past PhD Students already does it (a `**PhD:**` line), but that's worth double-checking once the actual information is in hand rather than assuming the format transfers directly.
- **Research Associates and Research Interns sections are empty**: both currently exist as headings with a description and no one listed underneath. Add current members as they're identified — Research Interns should use the no-photo entry pattern (Section 5), matching Past Research Interns.
- **Past People**: double-check whether anyone who has left ODEKA is missing from the appropriate "Past..." section.

11.2 Projects Page

Every project's actual page content is currently a placeholder ("Page under construction"). Beyond just writing that content, the following are genuinely open questions — not yet decided, and worth real thought rather than a quick default:

- What a project page should actually contain, and how it should be laid out.
- What the illustration strategy should be (one image per project? several? a gallery?).

- How to choose what image represents a given project.

One specific open question, unresolved as of this writing: Jon has asked for articles to be listed *by theme* somewhere on the site, but the current structure is organized by *project*, and a theme is not the same thing as a project (a single theme could span several projects, or a project could touch several themes). How — or whether — to reconcile "browse by project" with "browse by theme" hasn't been figured out yet.

It's worth being precise about what "project" means here in the first place: from ODEKA's side, what we've been calling a project is really an RCT program — a specific randomized evaluation, not a research topic. Organizing the site around that unit seems like the intuitive choice for what is, at bottom, a program-evaluation research platform: it's the natural unit of *our* work. But that intuition is exactly the kind worth distrusting a little — what's obvious from ODEKA's side of the platform isn't necessarily what's most useful to the researchers actually browsing it, and their sense of what should group together (by theme, by question, by region, by something else entirely) may not line up with the program structure at all. This is a real design question, not just a missing feature, and deserves to be thought through with that in mind rather than defaulted one way or the other.

11.3 News Page

The intended model for this section is that people out in the field write their own news posts and sign them in their own name — a first-person record of their time at ODEKA, not just an announcements feed. If someone in the field wants to do that, they should keep writing.

Getting credit for what you wrote, under your own name, is itself part of the motivation for anyone to want to keep this up — so the byline matters, not just the content. As of this writing, the existing posts are attributed to "The Research Associate Team" rather than the individual who actually wrote them (Adrien, for all of them). If the sign-your-own-name model above is the one that's actually adopted going forward, those existing posts' `author:` fields should be updated to match — back to the person who wrote each one — rather than left as a team byline while new posts are signed individually (Section 7).

If nobody is willing to keep the section updated, though, that's a real, acceptable option too: rather than let it visibly go stale, turn it off (Section 7.2). Nothing is lost by doing that — every post that's already been written stays intact and can be switched back on the moment someone wants to pick it back up.

11.4 Custom Domain, and Where the Address Should Land

The site currently lives at its default GitHub Pages address (`https://odeka-asbl.github.io`), not a domain ODEKA owns. Two separate things need deciding here, and they shouldn't be conflated:

- **Linking a custom domain.** This is mostly mechanical once a domain is chosen and registered: GitHub Pages supports it via a `CNAME` file naming the domain, plus a DNS record set up wherever the domain was purchased (either a `CNAME` record pointing at `odeka-asbl.github.io`, or `A` records pointing at GitHub's IP addresses, depending on whether it's a subdomain or an apex/root domain). The `CNAME` file itself should live in the *project root* (alongside `_quarto.yml`), not be placed by hand inside `docs/` — Quarto automatically copies loose files like this from the project root into `docs/` on every render (the same way it already does for `favicon.ico`), so a root-level copy survives renders reliably where a `docs/`-only one wouldn't.

Registering the domain itself and configuring DNS happens outside this repository, with whoever manages ODEKA's domain registrar — Claude Code can help with the `CNAME` file and the DNS record values once a domain is picked, but can't register a domain or log into a registrar on its own.

- **Where the address should land.** Separately, and still undecided: Jon has asked that the website address land directly on the *Projects* page rather than the current homepage. That's not automatic just from pointing a domain at the site — as things stand, the root address serves `index.qmd` (the homepage), and Projects is a page underneath it. Making Projects the landing page means an actual decision about *how*: whether the homepage should redirect to Projects, whether Projects should effectively *become* the homepage (absorbing whatever from the current homepage is still worth keeping — the project carousels, the quotes, the News section), or whether the navbar and homepage both stay as they are and only the bare domain root redirects. This is a real design question in the same spirit as the project-versus-theme one above (Section 11.2), not a small technical toggle — worth resolving deliberately once a domain is actually in hand, rather than picking the easiest option by default.

11.5 Keeping This Document Itself Up to Date

As items on this list get done — full names added, photos in place, a real project page written — this guide should be updated to match, not left describing a state of the site that no longer exists. That's exactly the kind of small, well-defined edit Claude Code is good at: point it at whatever changed and ask it to update the relevant part of `read_me/odeka_website_guide.tex` accordingly (it can even recompile the PDF for you).

The same goes for `CLAUDE.md` (Section 4.3): the two documents are meant to stay in sync. As a rough rule, if a fix or a decision would help a future *person* doing this work, it belongs here; if it would help a future *Claude Code session* avoid re-debugging the same problem, it belongs in `CLAUDE.md`. Often it's worth adding to both.